AI-Driven Monitoring and Predictive Maintenance

on Smart Home Media Server

Ryan Arnzen • Prasiddha Neupane

Most interesting finding

A LSTM model with a 30-minute window had the lowest test error, this shows that our recent server scrapes held enough short-term data to forecast the next 10-minute CPU average better than a longer 60-minute window or using a flattened MLP model baseline

0.0316

Best Test RMSE

30-minute LSTM

10,081

Rows

one week at 1-minute steps

0

Missing values

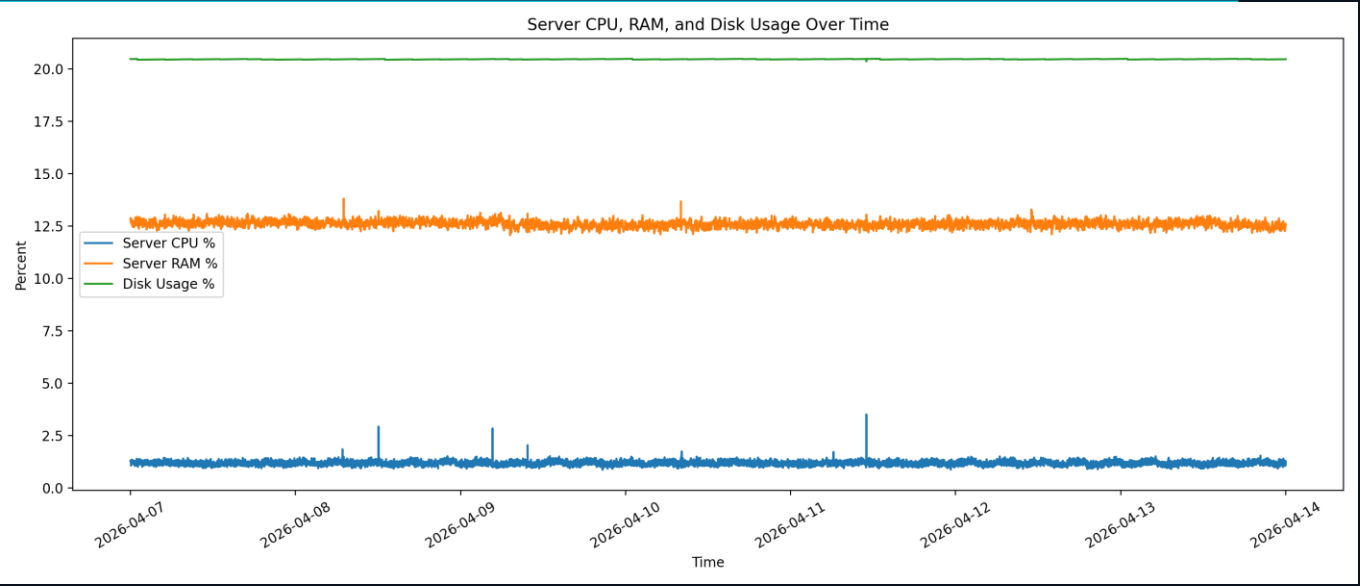
complete telemetry export with preprocessing

Problem and Motivation

Move from reactive monitoring to short predictive maintenance.

Smart home media servers run multiple services continuously: media streaming, cloud storage, reverse proxy, metrics, and container workloads. Waiting until CPU or memory is already high makes alerts reactive instead of predictive.

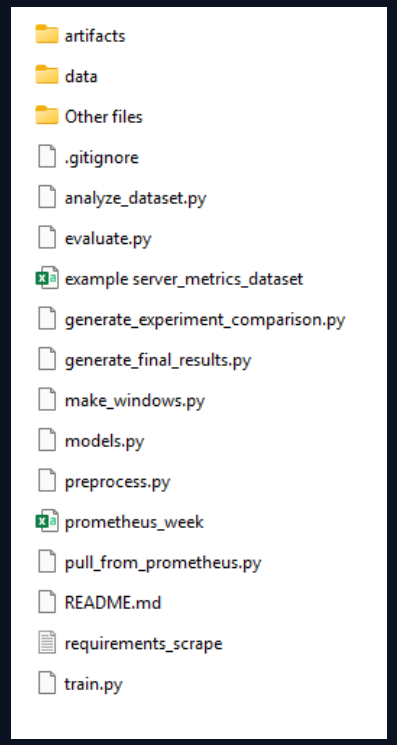
Goal: predict short-term CPU load from recent server and container telemetry.
Why CPU: it reflects system stress during transcoding, background services, and container activity.
Value: early warning, anomaly detection, and automated maintenance triggers.



End-to-End Approach

Deliverables in Zip file

Raw and processed CSV/NumPy datasets, metadata, and reusable scripts.
Dataset tables, seven visualizations, model checkpoints, learning curves, predictions, residuals, and final comparison charts.
Three experimental settings: MLP 60-min, LSTM 60-min, and LSTM 30-min.



Dataset Overview

One week of server telemetry collected from Prometheus at a one-minute interval.

10,081

Rows

2026-04-07 to 2026-04-14

7

Columns

timestamp + six metric columns

60 sec

Step

fixed interval after export

0

Missing

complete raw export

Raw columns and roles

```
The label is the future 10 minute average CPU value.
The other metrics are used as model input features.
```

timestamp	datetime64[UTC]	time index
server_cpu_percent	float64	target source
server_ram_percent	float64	feature
disk_usage_percent	float64	feature
container_ram_mb	float64	feature
container_cpu_cores	float64	feature
container_count	int64	feature

Methodology: Models and Learning Objective

Compared a flattened neural baseline to a LSTM recurrent model.

MLP baseline

Flatten(60×6) → Dense(64, ReLU) → Dense(32, ReLU) → Dense(1)
25,217 trainable parameters
Tests whether a non-sequential model can learn from recent metrics. Turns data into a list of numbers and outputs one predicted CPU value.

LSTM primary

LSTM(64) → Dropout(0.2) → Dense(16, ReLU) → Dense(1)
19,233 trainable parameters
Processes server telemetry in time order, preserving sequential data.

Training objective

Minimize MSE using Adam optimizer; report MSE, MAE, and RMSE on the held-out test split.

$$MSE = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$$
$$MAE = (1/n)\ \Sigma|y_i - \hat{y}_i|$$
$$RMSE = \sqrt{MSE}$$

Why this comparison: if the LSTM beats the MLP, the order of our readings is adding useful forecasting.

Dataset Statistics and Target Variable

The target is the future 10-minute average CPU usage.

Statistical snapshot

Metric	Mean	Std	Min	Max
CPU %	1.168	0.104	0.864	3.514
RAM %	12.611	0.149	12.067	13.818
Disk %	20.473	0.010	20.366	20.496
Container RAM MB	1749.203	21.196	1672.477	1831.941
Container CPU cores	0.082	0.006	0.063	0.107
Container count	10.000	0.000	10.000	10.000

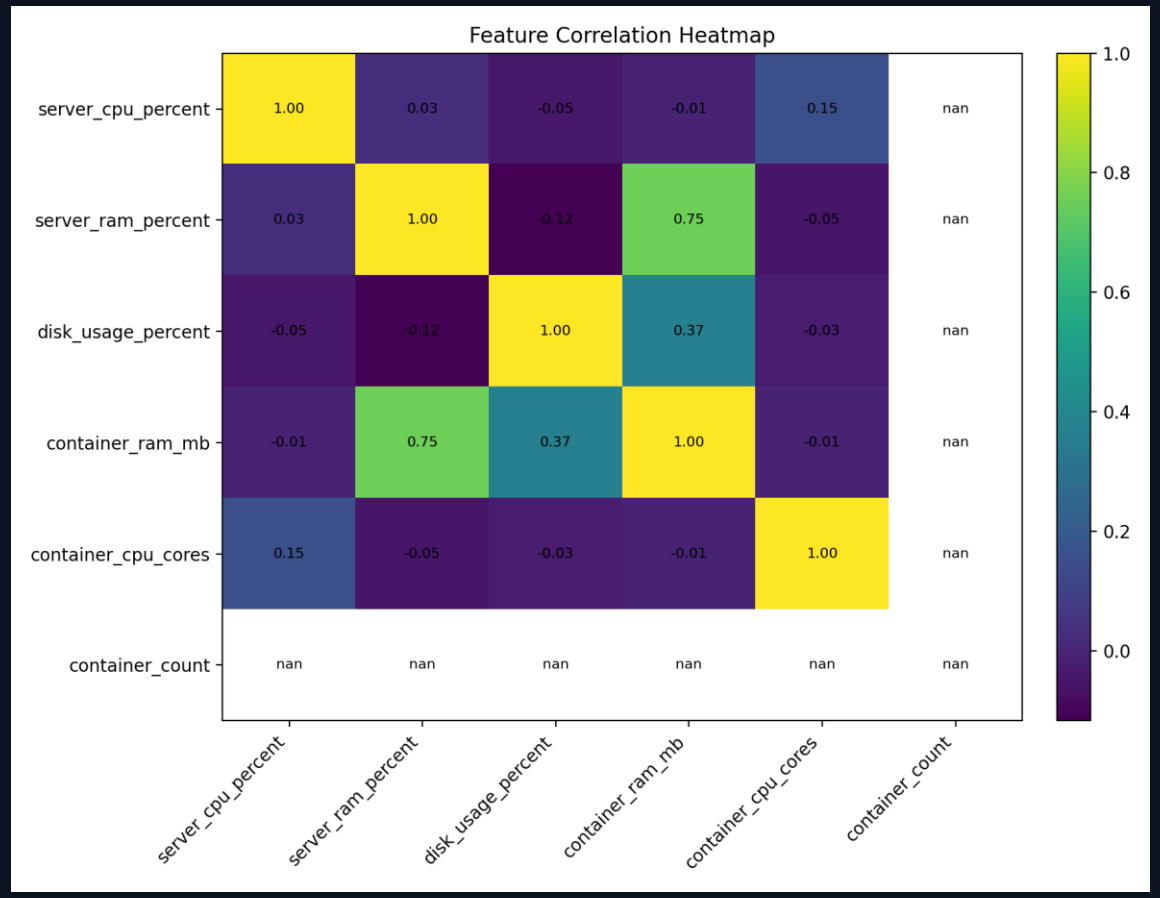
Target formula

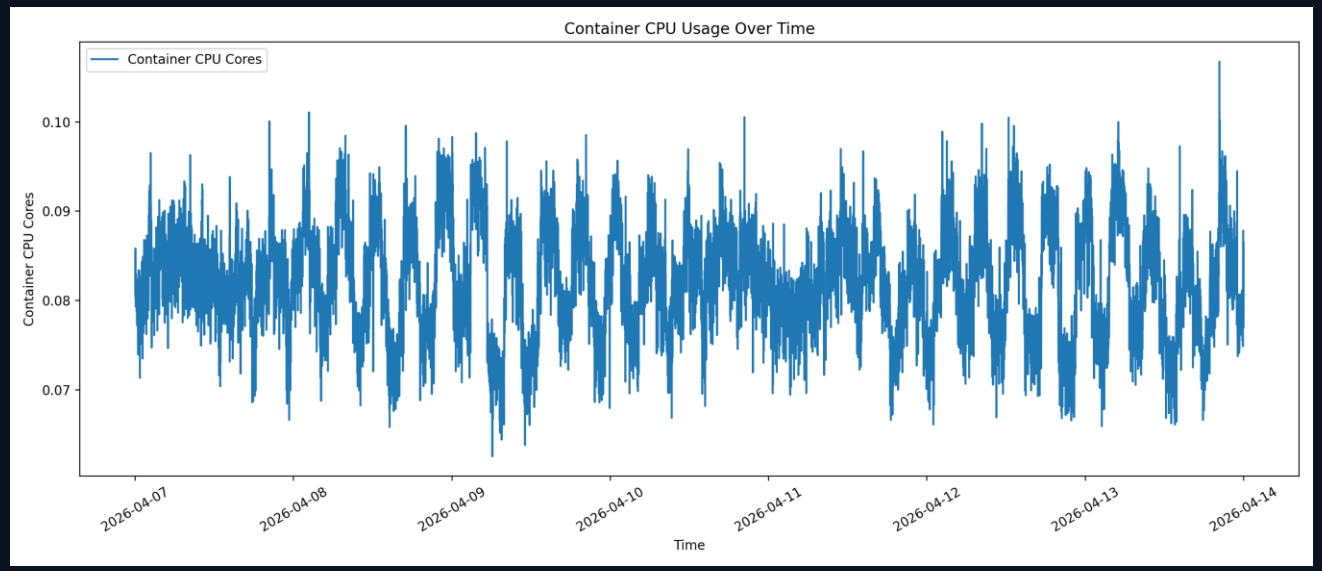
target_t = mean(cpu_{t+1}, ..., cpu_{t+10})

Our target is the average of the next 10 CPU readings. CPU stayed relatively low and stable, so the focus is on short term forecasting rather than high-load classification.
No missing values were present in the raw export; preprocessing still included fixed-interval resampling and forward/backward filling.

Feature Relationships

Correlation and container CPU plots help explain what the model has and hasn't learned.





Key reading

Container CPU has visible minute-to-minute movement that provides useful sequential data for our model.
Container count is constant, so it contributes no variance in this run.
Low variance in some of our features shows why RMSE values are small and why our future implementations need to test with higher server loads.

Experimental Setup

Time-ordered splits and training-set-only scaling avoid future-data leaks.

Experiment matrix

MLP baseline	MLP	60-min window	next 10-min avg CPU
LSTM primary	LSTM	60-min window	next 10-min avg CPU
LSTM shorter window	LSTM	30-min window	next 10-min avg CPU

7,051

Train rows

first 70% by time

1,510

Validation rows

middle 15% by time

1,510

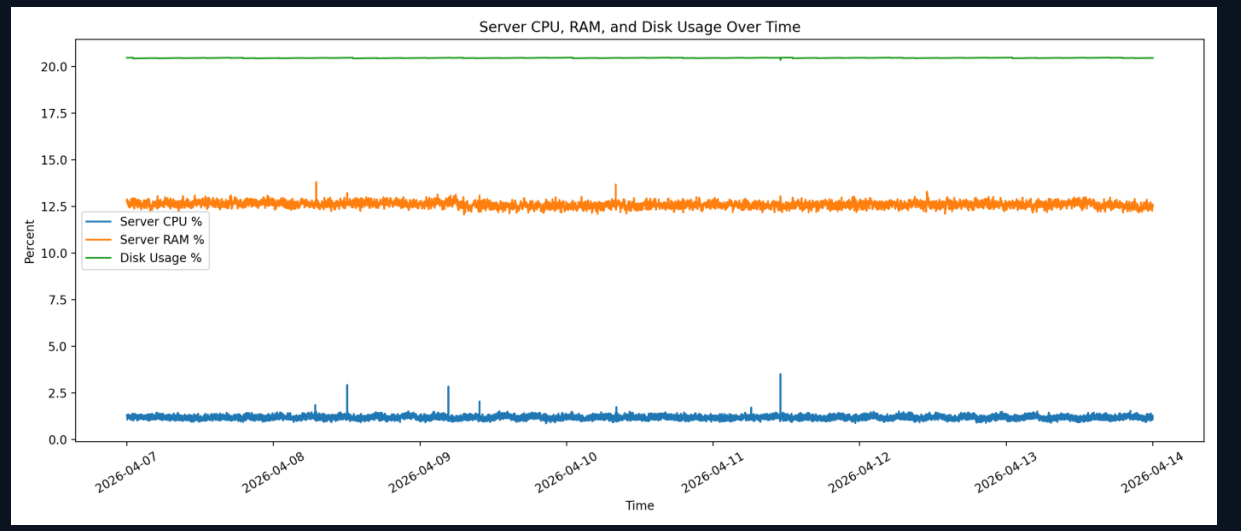
Test rows

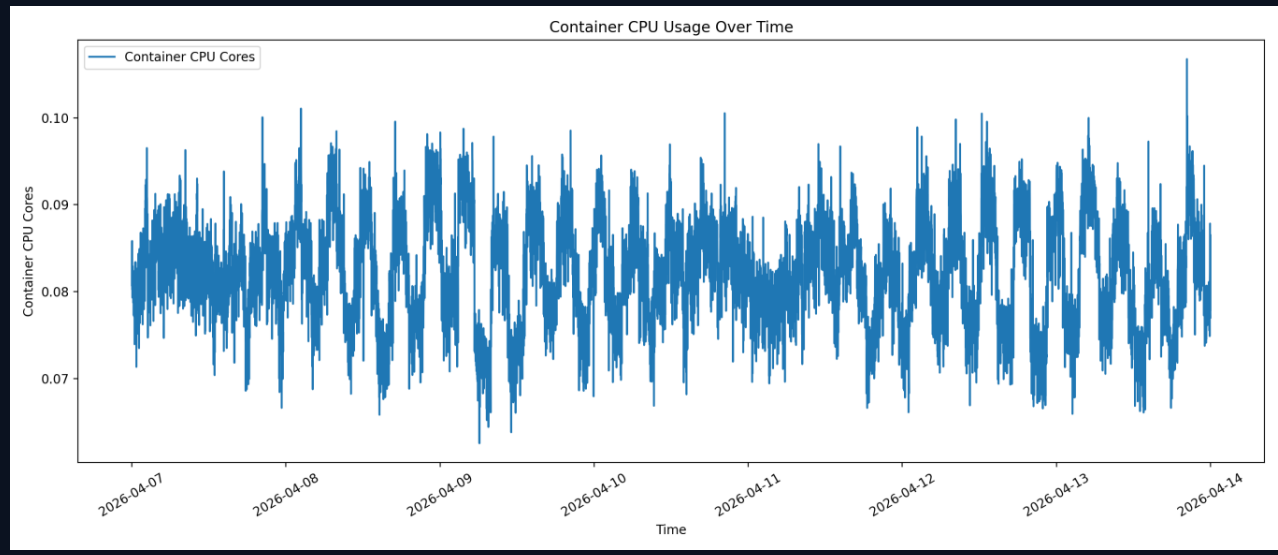
final 15% by time

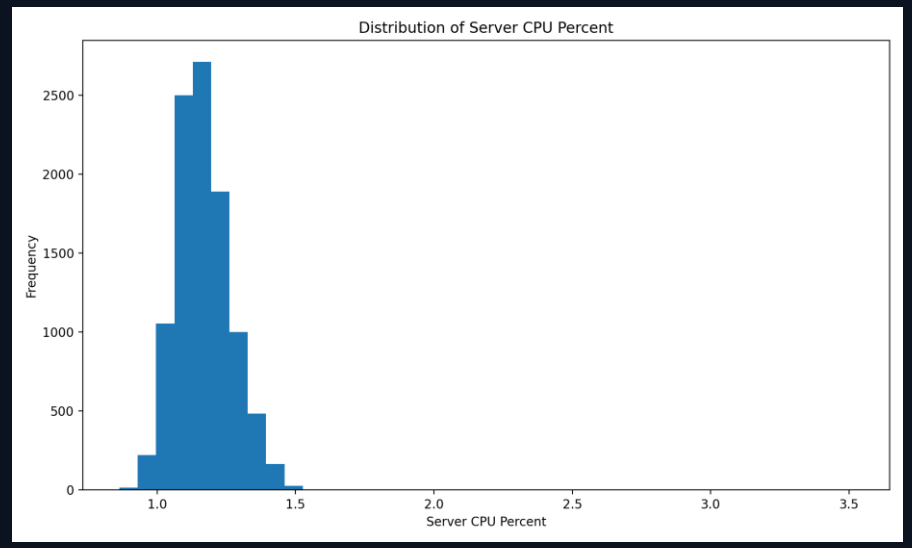
Features were standardized using training-set mean and standard deviation.
Early stopping monitored validation loss with patience of 5 epochs.
Best model checkpoints and final Keras models were saved per run.
Training ran on CPU because CUDA drivers were not available on the laptop.

Exploratory Data Analysis: System Load Over Time

EDA shows stable server behavior with small CPU spikes and light container RAM movement.





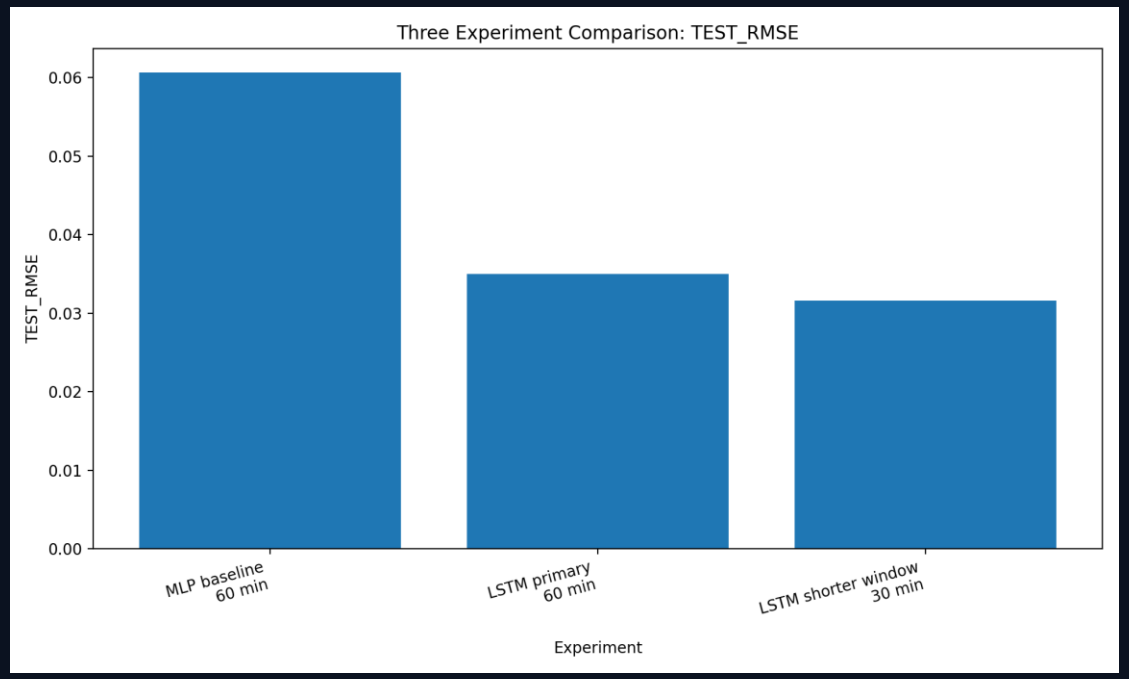


Interpretation

CPU was very stable and did not fluctuate drastically throughout the data that was collected.

Results: Performance Comparison

The LSTM models beat the MLP baseline; the 30-minute LSTM was best in the final experiment set.



Test metrics

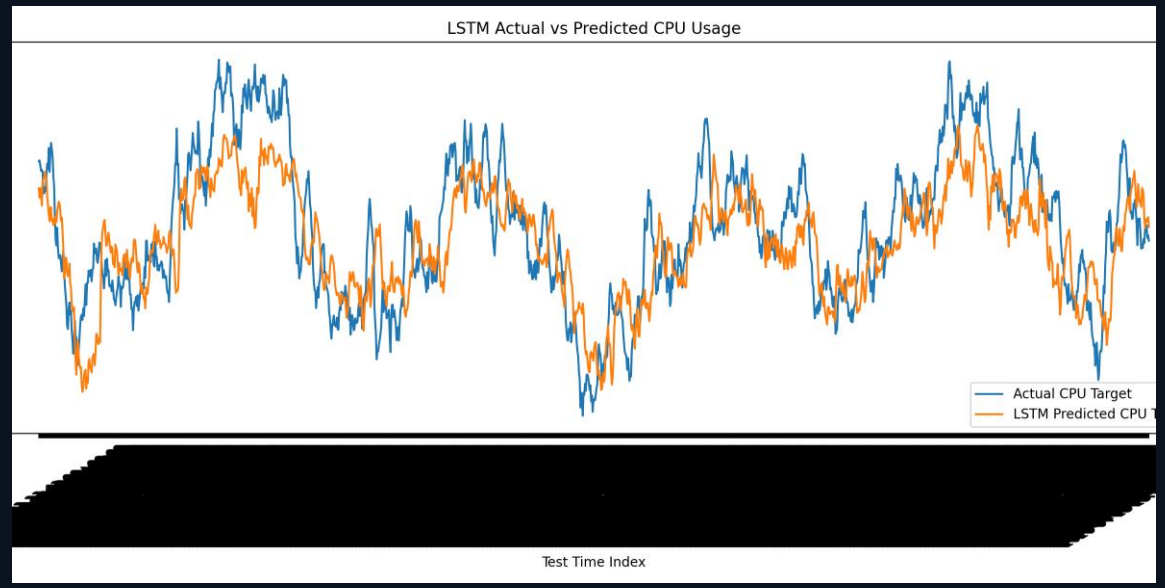
Experiment	MSE	MAE	RMSE
MLP 60-min	0.003684	0.048503	0.060694
LSTM 60-min	0.001227	0.028447	0.035034
LSTM 30-min	0.001000	0.026000	0.031600

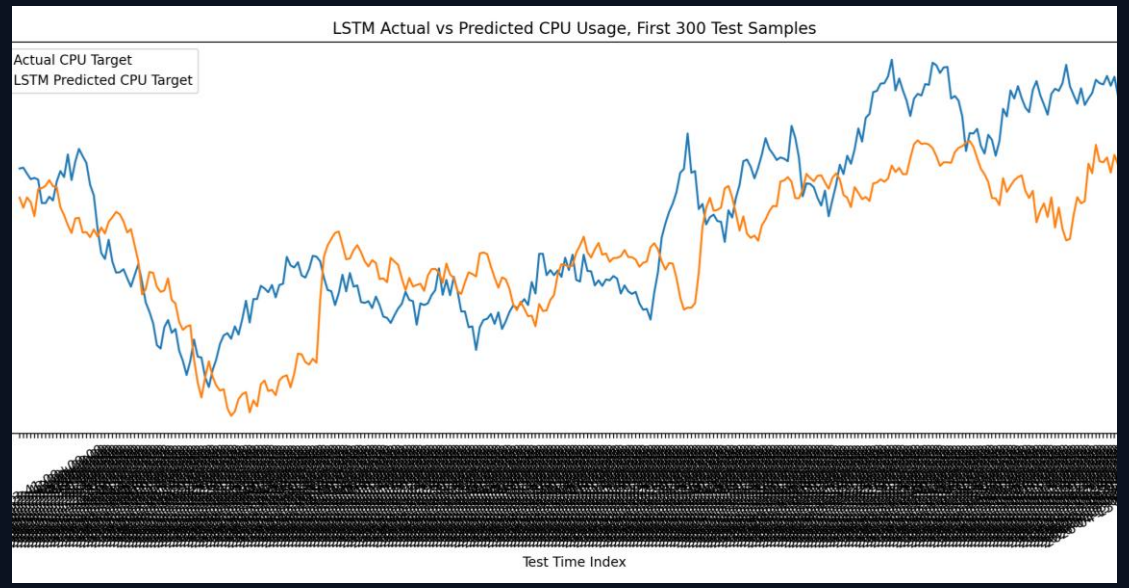
Conclusion

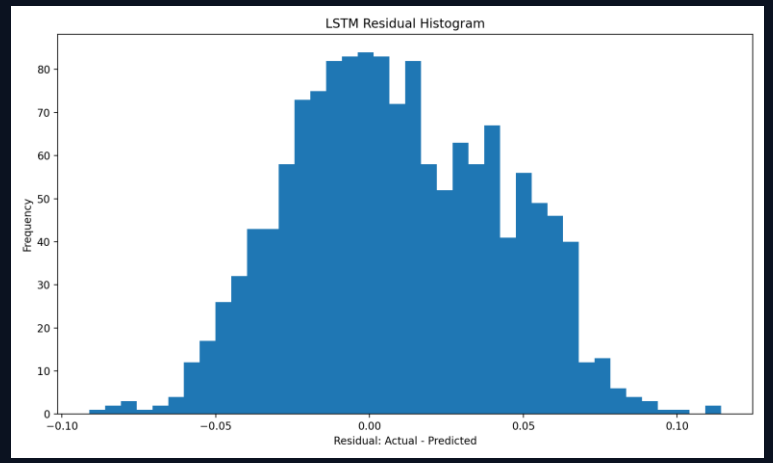
LSTM 60-min reduced RMSE by about 42% relative to the MLP baseline.
LSTM 30-min improved again, suggesting the most recent half hour was enough context for this stable workload.
The improvement supports modeling for short-term server telemetry prediction.

Results: LSTM Predictions and Residuals

Visual checks show the model tracks the low-load CPU pattern, with small residuals.







After training, the LSTM predictions generally track the actual held-out test telemetry pattern.

The set of predictions based on the first 300 samples had larger variations in predictions.

The residual histogram shows most prediction errors are distributed near zero. Most errors are within about ±0.05 CPU percentage points, and nearly all are within about ±0.10 CPU percentage points.

Discussion, Limitations, and Future Work

Positive results

End-to-end pipeline is complete: export, analyze, preprocess, window, train, evaluate, and save artifacts.
LSTM sequence modeling consistently improves over the flattened MLP baseline.
The project uses server telemetry collected from the home media environment.

Limitations

Only one week of data was tested, and CPU load was mostly low.
The target is CPU forecasting, not full predictive maintenance classification yet.
Container count did not vary, limiting feature diversity.

Next improvements

Collect longer time spans and including high-load events: Plex transcoding, backups, downloads, and simulated stress tests.
Turn forecasts into decisions: anomaly score, overload warning, and maintenance action.
Deploy the model as a container for real time data analysis.

Individual Contributions and References

Final slide for accountability, sources, and Q&A.

Ryan Arnzen

- Server architecture and integration
- Data collection pipeline
- Feature engineering
- Model implementation baseline
- Git Documentation

Prasiddha Neupane

- Exploratory data analysis
- Model evaluation and comparison
- Performance analysis and visualization
- Documentation and final report preparation

References / tools used

Tools:
Git, Python, NumPy, pandas, matplotlib, TensorFlow/Keras.
Prometheus HTTP API, and Prometheus telemetry export.

GitHub repo: RyArnz/csc426-smart-media-ml-monitoring.
TensorFlow/Keras: https://www.tensorflow.org/-
pandas: https://pandas.pydata.org/
NumPy: https://numpy.org/
matplotlib: https://matplotlib.org/
Prometheus HTTP API: https://prometheus.io/docs/prometheus/latest/querying/api/

QUESTIONS?